

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №9
по дисциплине «Вычислительная математика»
Тема: Составные формулы прямоугольников, трапеций, Симпсона.

Студент гр. 4342

Закаблук А.Ю.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Цель лабораторной работы — изучить численное вычисление определённого интеграла с помощью квадратурных формул прямоугольников, трапеций и Симпсона. В ходе работы требуется реализовать вычисление заданной подынтегральной функции и программные функции для нахождения интеграла каждым из указанных методов. Также необходимо применить правило Рунге для оценки погрешности и подобрать наименьшее значение n , то есть наибольшее значение шага h , при котором приближённое значение интеграла будет найдено с заданной точностью ε . Дополнительно требуется сравнить работу формул прямоугольников, трапеций и Симпсона по полученным значениям интеграла, числу разбиений n и точности результата.

Задание.

Повышения точности численного интегрирования добиваются путем применения составных формул. Для этого при нахождении определенного интеграла отрезок $[a, b]$ разбивают на четное $n = 2m$ число отрезков длины $h = (b - a) / n$ и на каждом из отрезков длины $2h$ применяют соответствующую формулу. Таким образом получают составные формулы прямоугольников, трапеций и Симпсона.

На сетке $x_i = a + i \cdot h$, $y = f(x_i)$, $i = 0, 1, 2, \dots, 2m$, составные формулы имеют следующий вид:

Формула прямоугольников (средних)

$$\int_a^b f(x) dx = h \sum_{i=0}^{n-1} f(x_i + h/2) + R_1$$

Формула трапеций

$$\int_a^b f(x) dx = \frac{h}{2} \left(f(a) + f(b) + 2 \sum_{i=1}^{n-1} f(x_i) \right) + R_2$$

Формула Симпсона (парабол)

$$\int_a^b f(x) dx = \frac{h}{3} \left(f(a) + f(b) + 4 \sum_{i=1}^m f(x_{2i-1}) + 2 \sum_{i=1}^{m-1} f(x_{2i}) \right) + R_3$$

($n=2m$ - четное число отрезков)

где R_1, R_2, R_3 — остаточные члены.

Для практической оценки погрешности квадратурной формулы можно использовать правило Рунге. Для этого проводят вычисления на сетках с шагом h и $h/2$, получают приближенные значения интеграла I_h и $I_{h/2}$ и за окончательные значения интеграла принимают величины:

$I = I_{h/2} + (I_{h/2} - I_h) / 3$ — для формулы прямоугольников;

$I = I_{h/2} + (I_{h/2} - I_h) / 3$ — для формулы трапеций;

$I = I_{h/2} + (I_{h/2} - I_h) / 15$ — для формулы Симпсона.

За погрешность приближенного значения интеграла для формул прямоугольников и трапеций тогда принимают величину: $|I_{h/2} - I_h| / 3$, а для формулы Симпсона: $|I_{h/2} - I_h| / 15$.

В лабораторной работе требуется, используя квадратурные формулы прямоугольников, трапеций и Симпсона, вычислить значения заданного

интеграла и, применив правило Рунге, найти наименьшее значение n , то есть наибольшее значение шага h , при котором каждая из указанных формул дает приближенное значение интеграла с погрешностью ε , не превышающей заданную.

Варианты заданий приведены в таблице 4.1. $\varepsilon = 0.01; 0.001; 0.0001$.

Вариант 5: $f(x) = \int_0^1 \exp(\cos x) dx$

Выполнение работы.

1. Составить программы-функции для вычисления интегралов по формулам прямоугольников, трапеций и Симпсона.

На первом этапе выполнения лабораторной работы были реализованы три отдельные программы-функции для численного вычисления определенного интеграла. Разделение на отдельные функции сделано для того, чтобы каждый метод можно было вызывать независимо и затем объективно сравнивать полученные результаты. Для всех трех методов используется общий подход: отрезок интегрирования $[a,b]$ разбивается на n равных частей, после чего вычисляется шаг сетки: $h=(b-a)/n$.

- **В функции для метода прямоугольников** реализована составная формула средних прямоугольников. На каждом малом отрезке выбирается его середина, в этой точке вычисляется значение подынтегральной функции, затем все найденные значения суммируются и умножаются на шаг h . Этот способ геометрически заменяет площадь под графиком функции суммой площадей прямоугольников.

- **В функции для метода трапеций** реализована составная формула трапеций. В этом методе на каждом малом отрезке график функции приближенно заменяется прямой линией (хордой). Значения функции в крайних точках всего промежутка $[a,b]$ учитываются с коэффициентом $1/2$, а значения во внутренних узлах сетки — с коэффициентом 1 . Результат суммирования умножается на шаг h .

- **В функции для метода Симпсона** реализована составная формула парабол. Данный метод использует более точное приближение: на парах соседних отрезков график функции заменяется параболой. Внутренние значения функции учитываются с чередующимися коэффициентами 4 и 2 , а значения на концах отрезка — с коэффициентом 1 . Так как формула применяется на парах отрезков, в программе строго контролируется, чтобы число разбиений n всегда было четным.

2. Составить программу-функцию для вычисления подынтегральной функции

По условию варианта 5 требуется вычислить определенный интеграл:

$$\int_0^1 \exp(\cos x) dx$$

Следовательно, подынтегральная функция имеет вид:

$$f(x)=e^{\cos(x)}.$$

Для её вычисления в программе была реализована отдельная функция $f(x)$, которая получает на вход аргумент x и возвращает `std::exp(std::cos(x))`. Функция является непрерывной, гладкой и не имеет разрывов на заданном отрезке интегрирования $[0,1]$, что гарантирует корректную работу квадратурных формул. Выделение функции в отдельный блок делает код универсальным: методы интегрирования не зависят от конкретного вида функции.

3. Составить главную программу, содержащую оценку по Рунге погрешности квадратурных формул.

На третьем этапе была составлена головная программа, выполняющая автоматический подбор количества разбиений n для достижения требуемой точности. В программе задаются пределы интегрирования ($a=0$, $b=1$) и требуемые точности ($\varepsilon=0.01;0.001;0.0001$).

Проверка точности выполняется с помощью правила Рунге. Сначала интеграл вычисляется на сетке с n отрезками (I_n), затем шаг сетки уменьшается вдвое (число отрезков становится $2n$), и вычисляется новое значение (I_{2n}). Различие в оценке погрешности связано с порядком точности методов (p):

Для формул прямоугольников и трапеций оценка погрешности вычисляется как:

$$|I(2n) - I(n)| / 3$$

Для формулы Симпсона оценка погрешности вычисляется как:

$$|I(2n) - I(n)| / 15$$

Различие в знаменателях связано с порядком точности методов. Формулы прямоугольников и трапеций имеют 2-й порядок точности ($p=2$), поэтому

знаменатель в правиле Рунге равен $2^2-1=3$. Формула Симпсона имеет 4-й порядок точности ($p=4$), поэтому знаменатель равен $2^4-1=15$.

Если найденная оценка больше заданного ε , программа удваивает n и повторяет процесс. Как только погрешность становится $\leq \varepsilon$, цикл останавливается.

4. Провести вычисления по программе, добиваясь, чтобы результат удовлетворял требуемой точности

На четвертом этапе были проведены вычисления. Для каждой формулы программа начинала с малого числа разбиений ($n=2$) и последовательно его удваивала.

Вычисления проводились для трех значений точности: $\varepsilon = 0.01$, $\varepsilon = 0.001$, $\varepsilon = 0.0001$

Для каждой формулы программа начинала вычисления с малого количества разбиений n , затем последовательно удваивала это значение. На каждом шаге интеграл вычислялся дважды: сначала при текущем значении n , затем при удвоенном количестве разбиений. После этого по правилу Рунге оценивалась погрешность результата.

Если оценка погрешности была больше заданного значения ε , количество разбиений снова увеличивалось в два раза. Процесс продолжался до тех пор, пока оценка погрешности не становилась меньше или равной заданной точности.

В результате были получены следующие значения:

ε	Метод	n	h	Значение интеграла	Оценка погрешности
0.01	Прямоугольники	4	0.25	2.3453258525	0.0037346618
0.01	Трапеции	4	0.25	2.3341334691	0.0074640058
0.01	Симпсон	4	0.25	2.3415950580	0.0000006240
0.001	Прямоугольники	8	0.125	2.3425126131	0.0009377464
0.001	Трапеции	16	0.0625	2.3411153723	0.0004618770
0.001	Симпсон	4	0.25	2.3415950580	0.0000006240

0.0001	Прямоугольники	32	0.03125	2.3416335198	0.0000586411
0.0001	Трапеции	64	0.015625	2.3415456221	0.0000288674
0.0001	Симпсон	4	0.25	2.3415950580	0.0000006240

По результатам вычислений видно, что все три метода позволяют получить приближенное значение интеграла с заданной точностью. При этом формула Симпсона достигает требуемой точности уже при меньшем количестве разбиений. Это объясняется тем, что формула Симпсона имеет более высокий порядок точности по сравнению с формулами прямоугольников и трапеций.

Для наиболее строгой точности $\varepsilon = 0.0001$ формулы прямоугольников и трапеций потребовали $n = 32$, а формула Симпсона уже при $n = 4$ дала погрешность меньше заданной. Следовательно, в данной задаче формула Симпсона оказалась наиболее эффективной.

5. Результаты работы оформить в виде краткого отчета, содержащего сравнительную оценку применяемых для вычисления формул.

В ходе выполнения лабораторной работы было проведено численное вычисление интеграла с использованием трех составных квадратурных формул. Главным критерием сравнения является количество разбиений n , которое потребовалось для достижения заданной погрешности. Чем меньше n , тем меньше вычислений подынтегральной функции $f(x)$ нужно выполнить программе, следовательно, тем эффективнее метод.

Сравнение значений n можно представить следующим образом:

Точность ε_{ps}	Прямоугольники	Трапеции	Симпсон
0.01	$n = 4$	$n = 4$	$n = 4$
0.001	$n = 8$	$n = 16$	$n = 4$
0.0001	$n = 32$	$n = 64$	$n = 4$

Из таблицы видно, что при ужесточении требования к точности методы прямоугольников и трапеций требуют значительного (экспоненциального)

увеличения числа разбиений. У формулы Симпсона число разбиений во всех трех случаях осталось равным 4.

Также можно сравнить оценки погрешности при одинаковом значении $n = 4$ для точности $\epsilon_{ps} = 0.01$:

Метод	n	Оценка погрешности
Прямоугольники	4	0.0037346618
Трапеции	4	0.0074640058
Симпсон	4	0.0000006240

По этим данным видно, что формула Симпсона дает ошибку в разы меньше, чем формула прямоугольников и формула трапеций. Это связано с тем, что формула Симпсона имеет более высокий порядок точности. Формулы прямоугольников и трапеций имеют второй порядок точности, а формула Симпсона — четвертый порядок точности. Поэтому при уменьшении шага сетки результат по формуле Симпсона уточняется быстрее.

Интересно также сравнить сами приближенные значения интеграла. При самой строгой точности $\epsilon_{ps} = 0.0001$ были получены следующие уточненные значения по Рунге:

Метод	I с уточнением по Рунге
Прямоугольники	2.3415748786
Трапеции	2.3415744896
Симпсон	2.3415944340

До применения поправки Рунге метод прямоугольников дал значение 2.34163, а метод трапеций — 2.34154. То есть методы приближаются к результату с разных сторон графика функции. Уточненные значения по Рунге практически совпадают между собой, что говорит о правильности расчетов и сходимости алгоритмов к истинному значению интеграла.

Дальнейшее увеличение

nn

для формулы Симпсона в рамках заданных точностей не требуется, так как ее оценка погрешности 0.0000006 значительно меньше самого строгого порога $\epsilon=0.0001$.

Вывод.

В ходе лабораторной работы были изучены составные квадратурные формулы для численного вычисления определенного интеграла на примере

$$\int_0^1 \exp(\cos x) dx$$

Были реализованы подпрограмма вычисления функции, а также составные формулы прямоугольников, трапеций и Симпсона. Для оценки точности применялось правило Рунге с автоматическим удвоением числа отрезков n .

По результатам установлено, что эффективность формул кардинально различается. Из методов 2-го порядка метод прямоугольников оказался эффективнее метода трапеций (потребовал в 2 раза меньше разбиений для точности 10^{-4}). Однако абсолютным лидером стала формула Симпсона (4-й порядок точности), которая во всех рассмотренных случаях достигла требуемой точности уже при $n=4$. Это подтверждает тот факт, что для вычисления определенных интегралов от гладких неразрывных функций применение методов высоких порядков позволяет колоссально экономить вычислительные ресурсы процессора.

ПРИЛОЖЕНИЕ А

Название файла: main.cpp

```
#include <iostream>
#include <cmath>
#include <iomanip>
#include <functional>
#include <vector>
#include <string>
#include <windows.h> // Для кодировки консоли

/*
    Подынтегральная функция.
    По условию варианта 5:
     $f(x) = \exp(\cos(x))$ 
    На отрезке [0; 1]
*/
double f(double x) {
    return std::exp(std::cos(x));
}

/*
    Составная формула прямоугольников (средних).
     $I \approx h * \text{сумма } f(x_i + h/2)$ 
*/
double rectangleFormula(double a, double b, int n) {
    double h = (b - a) / n;
    double sum = 0.0;
    for (int i = 0; i < n; ++i) {
        double x = a + (i + 0.5) * h; // Середина отрезка
        sum += f(x);
    }
    return h * sum;
}

/*
    Составная формула трапеций.
     $I \approx h * ( (f(a) + f(b)) / 2 + \text{сумма } f(x_i) )$ 
*/
double trapezoidFormula(double a, double b, int n) {
    double h = (b - a) / n;
    double sum = (f(a) + f(b)) / 2.0;
    for (int i = 1; i < n; ++i) {
        double x = a + i * h;
        sum += f(x);
    }
    return h * sum;
}

/*
    Составная формула Симпсона.
     $I \approx h/3 * (f(x_0) + 4*f(x_1) + 2*f(x_2) + \dots + f(x_n))$ 
    Внимание: n должно быть четным!
*/
double simpsonFormula(double a, double b, int n) {
    if (n % 2 != 0) ++n; // Гарантируем четность
```

```

double h = (b - a) / n;
double sum = f(a) + f(b);

for (int i = 1; i < n; ++i) {
    double x = a + i * h;
    if (i % 2 == 1) {
        sum += 4.0 * f(x); // Нечетные индексы
    } else {
        sum += 2.0 * f(x); // Четные индексы
    }
}
return (h / 3.0) * sum;
}

/*
Структура для хранения результата работы метода.
*/
struct Result {
    std::string methodName;
    double eps;
    int n;
    double h;
    double integral;
    double rungeIntegral;
    double error;
    int iterations;
};

/*
Универсальная функция для применения правила Рунге.
*/
Result rungeRule(
    const std::string& methodName,
    const std::function<double(double, double, int)>& formula,
    int order, // Порядок точности (2 для прямоугол/трапеций, 4 для
Симпсона)
    double a,
    double b,
    double eps
) {
    int n = 2; // Начинаем с n=2
    double oldIntegral = formula(a, b, n);
    int iterations = 0;

    while (true) {
        n *= 2; // Удваиваем сетку
        ++iterations;

        double newIntegral = formula(a, b, n);

        // Знаменатель в правиле Рунге: 2^p - 1
        double denominator = std::pow(2.0, order) - 1.0;

        // Оценка погрешности
        double error = std::abs(newIntegral - oldIntegral) / denominator;

        // Уточненное значение по Рунге (с учетом знака ошибки)
        double signedError = (newIntegral - oldIntegral) / denominator;

```

```

double rungeIntegral = newIntegral + signedError;

if (error <= eps) {
    Result result;
    result.methodName = methodName;
    result.eps = eps;
    result.n = n;
    result.h = (b - a) / n;
    result.integral = newIntegral;
    result.rungeIntegral = rungeIntegral;
    result.error = error;
    result.iterations = iterations;
    return result;
}
oldIntegral = newIntegral;
}
}

// Функция вывода
void printResult(const Result& result) {
    std::cout << std::left << std::setw(20) << result.methodName
        << "| eps: " << std::setw(6) << result.eps
        << "| n: " << std::setw(3) << result.n
        << "| h: " << std::setw(8) << result.h
        << "| I = " << std::setw(12) << result.integral
        << "| I (Пунре) = " << std::setw(12) << result.rungeIntegral
        << "| Err: " << std::setw(12) << result.error << "\n";
}

int main() {
    // Настраиваем вывод консоли на UTF-8 для корректного русского языка
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);

    std::cout << std::fixed << std::setprecision(10);

    // Параметры интегрирования
    double a = 0.0;
    double b = 1.0;
    std::vector<double> epsValues = {0.01, 0.001, 0.0001};

    std::cout << "Лабораторная работа №9\n";
    std::cout << "Интеграл: exp(cos(x)) dx на отрезке [0; 1]\n";
    std::cout << std::string(110, '-') << "\n\n";

    for (double eps : epsValues) {
        std::cout << ">>> Требуемая точность eps = " << eps << "\n";

        Result rRect = rungeRule("Прямоугольники", rectangleFormula, 2,
a, b, eps);
        Result rTrap = rungeRule("Трапеции", trapezoidFormula, 2, a, b,
eps);
        Result rSimp = rungeRule("Симпсон", simpsonFormula, 4, a, b,
eps);

        printResult(rRect);
        printResult(rTrap);
        printResult(rSimp);
    }
}

```

```
        std::cout << std::string(110, '-') << "\n";  
    }  
    return 0;  
}
```